

Trabajo Práctico Especial: Zephyr OS vs MOSIX — Análisis Comparativo

Universidad Nacional de Mar del Plata — Facultad de Ingeniería Materia: Fundamentos de Sistemas Operativos (2026) **Grupo:** ARRIAGA, Mario Esteban · BELLONE, Martín · BISCAY, Federico Javier · CALLA ALIENDE, Federico · CARDOZO, Lucas **Fecha de entrega:** 3 de Junio de 2026

Índice

1. Introducción
 2. La Empresa
 3. Características Generales
 4. Sistema de Archivos
 5. Administración de Memoria
 6. Administración del Procesador
 7. Seguridad
 8. Facilidades para Desarrolladores
 9. Puertas Afuera
 10. Comparativa Técnica
 11. Conclusiones y Recomendaciones
 12. Bibliografía
-

1. Introducción

Este informe compara **Zephyr OS** (RTOS para IoT embebido, ~16 KB RAM mínimo, Linux Foundation) y **MOSIX** (sistema de clustering HPC nacido en 1977 en la Hebrew University of Jerusalem). Aunque responden a problemáticas radicalmente distintas —dispositivos restringidos vs. clusters de cientos de nodos—, la comparación ilustra cómo diferentes dominios generan soluciones arquitectónicas opuestas.

2. La Empresa

2.1 Zephyr OS — Linux Foundation

Origen: Febrero 2016. Wind River donó el kernel de Rocket RTOS (originado en Virtuoso RTOS, adquirido a Eonic Systems en 2001) a la Linux Foundation. Miembros fundadores: Intel, Wind River, Synopsys, NXP.

Gobernanza: Technical Steering Committee (TSC) + Governing Board con representantes de miembros corporativos.

Respaldo (2025-2026): Platinum/Gold: Qualcomm, CARIAD (VW), Renesas, ZEISS, Analog Devices, Silicon Labs, Wind River, Antmicro. Silver: Nordic, Google, Meta, STMicroelectronics,

Texas Instruments, Espressif, Arduino, Canonical, Microchip, Infineon.

Mercado: IoT, microcontroladores de 32-bit (~16 KB RAM mínimo), wearables, industrial, dispositivos médicos, transporte, energía renovable.

Modelo: Código abierto, Apache 2.0, 3,000+ contribuyentes, 1,000+ boards (ARM Cortex-M, RISC-V, x86, ARC).

2.2 MOSIX — Hebrew University of Jerusalem

Origen: 1977, Prof. Amnon Barak. Cronología: 1977-1979: PDP-11/45 con Unix v6 (primera migración de procesos). 1981-1983: MOS (antecesor), cluster de 5 PDP-11. 1988-1989: primer “MOSIX” de 16 nodos. 1999: transición a Linux. 2001: se vuelve propietario. 2002: openMosix (fork GPL, discontinuado 2008). 2014: MOSIX-4 (funciona como módulo sin parche de kernel). Oct 2017: último release MOSIX-4.4.4.

Gobernanza: Investigador principal único. Sin Governing Board. MOSIX® marca registrada.

Respaldo: Académico (71+ publicaciones, ~1,662 citas). Sin soporte comercial.

Mercado: HPC, clusters científicos, grids, Single System Image (SSI).

Modelo: Propietario restrictivo (prohíbe modificación, ingeniería reversa, obras derivadas). No es open source.

Síntesis

Aspecto	Zephyr OS	MOSIX
Organización	Linux Foundation	Hebrew University of Jerusalem
Origen	2016	1977
Tipo	Código abierto comercial	Investigación académica
Licencia	Apache 2.0 (permisiva)	Propietaria restrictiva
Segmento	IoT, embebidos, wearables	HPC, clusters, grids
Estado	Activo (LTS3, 2026)	Inactivo desde 2017

3. Características Generales

Aspecto	Zephyr OS	MOSIX
Tipo de kernel	Híbrido monolítico configurable	Extensión de kernel Linux (módulo + daemon)
Arquitectura	Single Address Space	SSI — cluster como único sistema
Modelo	RTOS para dispositivos embebidos	Sistema operativo distribuido para clusters
Footprint mínimo	~16 KB RAM (nanokernel histórico ~4 KB)	N/A (requiere máquinas Linux completas)

Aspecto	Zephyr OS	MOSIX
Protección	MPU (8-16 regiones)	Protección nativa de Linux por nodo

Zephyr OS: Kernel monolítico unificado (desde v1.6). Single Address Space: syscalls son llamadas a función C directas sin trap ni cambio de contexto. Scheduling: cooperative + preemptive priority-based. Soporte AMP via OpenAMP.

MOSIX: Capa sobre kernel Linux existente. Módulo de kernel que intercepta syscalls + daemon en espacio de usuario. Paradigma: **migración preemptiva de procesos** (mover proceso en ejecución de un nodo a otro transparentemente). **Memory Ushering:** migración proactiva de memoria antes de OOM (swapping distribuido a nivel de cluster). Modelo **shared-nothing:** cada nodo memoria local independiente. No soporta threads ni memoria compartida entre nodos.

4. Sistema de Archivos

Aspecto	Zephyr OS	MOSIX
Arquitectura	VFS (capa de abstracción central)	DFSA (Distributed File System Adapter)
FS propios	LittleFS, FAT FS, NVS	No posee — delega a FS locales
FS subyacentes	Flash interna, SD card, USB	ext3, ext4, XFS, NFS, ext2
Permisos UNIX	No	POSIX completo

Zephyr OS: VFS estilo POSIX pero **no es POSIX compliant** (faltan `fcntl()`, `flock()`, `mmap()`). Tres FS:

- **LittleFS:** log-structured para flash interna, write-always (nunca sobrescribe), garbage collection, wear leveling automático, tolerancia a power loss (transacciones atómicas + CRC). RAM mínima ~2 KB (para el filesystem, no del OS).
- **FAT FS:** para SD y USB. Sin wear leveling, no tolerant a power loss. **No usar en flash interna.**
- **NVS:** clave-valor para configuración, datos por ID numérico, sin estructura de directorios, con wear leveling propio.

No soporta symbolic/hard links. Modelo de permisos inexistente (similar a DOS).

MOSIX: No provee FS distribuido propio. **DFSA** intercepta syscalls y redirige al nodo donde reside el archivo. Cada nodo mantiene su FS local. Limitaciones: no es parallel filesystem (archivo en un solo nodo), sin striping, latencia de red como cuello de botella, enlaces no cruzan límites de nodo.

5. Administración de Memoria

Zephyr OS

MPU vs MMU: Usa **MPU** en microcontroladores ARM Cortex-M y RISC-V embebido. Define 8-16 regiones con permisos separados (lectura/escritura/ejecución). **No traduce direcciones:** dirección virtual = física. Acceso fuera de región permitida genera excepción. MMU solo en plataformas avanzadas (Cortex-A, x86, RISC-V de aplicación) para memoria virtual y paginación.

Modelo de memoria plana: Single Address Space: kernel y apps comparten espacio. Aislamiento mediante **Memory Domains** (colección de particiones que define acceso por thread) y **Partitions** (regiones contiguas con atributos).

Regiones: KERNEL (modo privilegiado: .text, .rodata, .data/.bss, stack interrupciones), APPLICATION (user mode con MPU), DRAM/PERIFÉRICOS (heap, stacks, devices). Tipo Normal (con caché) o Device (sin caché).

Asignadores: **k_heap** (sincronizado multi-thread), **sys_heap** (sin sincronización, combina bloques adyacentes, buckets por tamaño, $O(1)$ 1-200 ciclos), Memory Slabs (tamaño fijo, $O(1)$), **k_malloc/k_free** (system heap, configurable, cero por defecto).

Sin swap: No hay swap ni memoria virtual en microcontroladores. Recursos definidos en compilación; thrashing no ocurre.

MOSIX

Modelo shared-nothing: Cada nodo memoria física independiente y autónoma. Sin memoria compartida entre nodos. Cuando agota memoria, migra procesos completos a otros nodos (no swapping a disco).

Memory Ushering: Detecta proactivamente nodos con memoria baja y migra procesos **antes** de contención severa. Flujo: monitoreo con umbrales críticos → identificación de nodo con memoria disponible → selección de proceso → transferencia por red (heap, stack, código, datos) → continuación en destino.

Checkpoint/Restart: Serializa estado completo: PCB (PID, estado, PC, registros, scheduling, descriptors), imagen de memoria completa. En destino: recrea espacio de direcciones, carga registros, reanuda ejecución.

Limitaciones: No soporta memoria compartida (ni POSIX **shm_open** ni System V **shmget/shmat**). Sin DSM. Procesos gigabytes toman minutos en migrar. debe evaluar si costo de migración supera beneficio.

Comparación

Aspecto	Zephyr OS	MOSIX
Hardware	Microcontroladores	Clusters de PCs
Protección	MPU (regiones fijas)	Aislamiento por nodo
Memoria virtual	Solo con MMU	No (migración de procesos)
Swap	No hay	No hay (migración)
Dinamicidad	Estática (compilación)	Dinámica (ejecución)
Thrashing	No ocurre	Memory Ushering previene

Aspecto	Zephyr OS	MOSIX
---------	-----------	-------

6. Administración del Procesador

Zephyr OS

Threads como unidad: Cada thread tiene propio stack y contexto, comparte espacio de direcciones con kernel y otros threads. `k_thread` = PCB simplificado (stack pointer, prioridad, estado, área de stack).

Estados: READY (en run queue), RUNNING (ejecutándose), SLEEPING (bloqueado), TERMINATED, SUSPENDED.

Políticas: Cooperative (prioridad negativa, retiene CPU hasta que libere voluntariamente) y Pre-emptive (prioridad no negativa, desalojo por prioridad mayor).

Priority-based sin Round Robin: Sin time-slicing, sin quantum. Determinismo > equidad. Consecuencias: starvation posible, sin fair sharing entre igual prioridad.

Syscalls como function calls: No hay `int 0x80` ni cambio de modo usuario/kernel. Thread de usuario llama función API wrapper → verifica punteros → transfiere control al kernel → retorna.

Priority inheritance: Evita inversión de prioridad (eleva temporalmente prioridad de hilo bloqueante).

MOSIX

Migración preemptiva como scheduler distribuido: Extiende administración a cluster completo. Ciclo: serializa checkpoint (PCB, memoria, archivos abiertos, sockets, señales) → transfiere por red → reconstruye en destino → continúa desde punto de serialización.

Load balancing multi-paramétrico: Evalúa simultáneamente velocidad de CPU, carga actual, memoria disponible, latencia de red, número de cores. Algoritmo adaptativo: nodos intercambian estadísticas, detectan desbalances, migran preemptivamente sin que la aplicación lo perciba.

Scheduler de tres niveles: Largo plazo (colocación inicial), medio plazo (memory ushering), corto plazo (evaluación continua de migración).

SSI: Cluster presentado como única máquina con N CPUs lógicas. Aplicaciones sin modificaciones ni recompilación.

Evitación del efecto convoy: Detecta procesos CPU-bound que monopolizan un nodo y los migra a nodos menos cargados.

Limitaciones: No soporta memoria compartida ni threads POSIX. Procesos grandes generan tráfico significativo.

Comparación

Aspecto	Zephyr OS	MOSIX
Unidad	Thread (contexto liviano)	Proceso completo (PCB)
Niveles scheduler Algoritmo	Solo corto plazo Priority-based puro	Largo + medio + corto Balanceo multi-paramétrico
Time slicing	No tiene	No tiene
Objetivo	Determinismo, latencia mínima	Throughput global de cluster

7. Seguridad

Zephyr OS

Aislamiento por MPU: Código/Flash: Read + Execute, No Write. RAM: Read + Write, No Execute. Periféricos: Read + Write, No Execute. Configuración solo en modo privilegiado (MRC/MCR en ARM, MSR en x86).

Modo dual: Kernel mode (privilegiado) y User mode (no privilegiado, restricciones MPU activas, trap en instrucciones privilegiadas).

ARM TrustZone: Secure World (criptografía, claves, root of trust) y Non-Secure World (aplicaciones normales).

Criptografía: PSA Crypto API con backend mbedTLS. TLS 1.2 mínimo obligatorio (versiones anteriores rechazadas por vulnerabilidades). Configuraciones predefinidas para TLS 1.2 con AES-CCM, CoAP/DTLS.

Secure Boot + MCUboot: Cadena: Hardware (RoT immutable) → ROM Bootloader → MCUboot → Zephyr Kernel + Apps. MCUboot usa RSA-2048, RSA-3072, ECDSA P-256. A/B partitioning para actualizaciones atómicas con rollback. Contador de imágenes previene rollback a versiones vulnerables.

Stack protection: Canaries entre buffers y return addresses (verificados antes de cada retorno/context switch). Regiones de stack no-ejecutables (MPU_XN).

Modelo de permisos: No DAC ni RBAC. Aislamiento: separación kernel/user via MPU, memory domains, filtrado de syscalls.

Certificaciones: OpenSSF Gold Badge (2018-03-10): 90% statement coverage, 80% branch coverage, auditoría NCC Group (2020), two-person code review. Gold Badge mantenido y actualizado hasta 2024-06-05.

MOSIX

Confianza mutua requerida: Todos los nodos deben ser mutuamente confiables. Sin mecanismo para verificar integridad de nodos ni aislar procesos de nodos no-confiables.

Sandboxing a nivel de kernel: LKM en modo privilegiado dentro del kernel de Linux. Intercepta syscalls → si son peligrosas, bloquea/redirige; si no, pasa al kernel normal.

Procesos guest:

Puede	No puede
Ejecutar código normalmente	Acceder archivos locales del nodo host
Usar CPU/memoria asignada	Modificar configuración del SO host
Acceder archivos del nodo origen (via DFSA)	Instalar software o cargar módulos
	Ver procesos del nodo host
	Syscalls peligrosas (mount, chroot, ptrace, syslog)

UID unificado (SSI): UID/GID numérico compartido entre nodos (UID “1000” en nodo A = UID “1000” en nodo B). Permisos calculados localmente.

Checkpoint/Restart: Transfiere UID efectivo/real, GIDs, capabilities, directorio de trabajo, descriptores, umask, límites, señales, estado de memoria. **Crítico:** archivos de checkpoint **no cifrados por defecto** — contienen toda la memoria (contraseñas, claves en texto claro).

Comparación

Aspecto	Zephyr OS	MOSIX
Aislamiento	Hardware (MPU)	Lógico (kernel module)
Criptografía	PSA Crypto + mbedTLS + Secure Boot	No disponible
Verificación firmware	Firmas asimétricas + rollback protection	No disponible
Control de acceso	MPU regions (sin DAC/RBAC)	UID unificado + permisos locales
Confianza en nodos	N/A (dispositivo único)	Requerida en todos
Multi-tenant	Compatible	Incompatible
Certificaciones	OpenSSF Gold Badge (2018-2024)	No aplica

8. Facilidades para Desarrolladores

Zephyr OS

West (meta-build tool): Gestiona repositorios Git múltiples, build, flash y debug en Python. `west init -l myapp`, `west update`, `west build -b <board>`, `west flash`, `west debug`.

CMake + KConfig: CMake genera archivos nativos (Ninja por defecto), cross-compilation nativa, multiplataforma, integración IDE. KConfig genera macros `#define CONFIG_X` para compilación condicional.

Device Tree: Archivos `.dts` describen periféricos, direcciones, pines, IRQs. Mismo driver funciona en Nordic nRF52840, STM32, ESP32 sin cambios. Flujo: selección de `.dts` → `dtc` genera `devicetree.h` con macros `DT_NODELABEL()`, `DT_PROP()` → driver abstrae hardware.

API POSIX-like: `open/close/read/write, socket/bind/listen/accept/connect, select/poll`. Por qué no POSIX completo: memoria limitada en microcontroladores, sin procesos separados (una única imagen con múltiples threads).

Zephyr SDK: GCC/Binutils/GDB, LLVM/Clang, QEMU (emulación), OpenOCD (debugging JTAG/SWD), Ninja.

Documentación: docs.zephyrproject.org — Getting Started, API Reference (Doxygen), Kernel Guide, Security, Samples. GitHub Discussions, Discord, mailing lists.

MOSIX

Compatibilidad POSIX total: No modifica interfaz de syscalls. Se inserta a nivel del scheduler, interceptando decisiones de migración después de que el kernel procesó las llamadas. Aplicaciones compilan/linkitan como en Linux normal. No requiere librerías especiales.

ELF estándar: Acepta ejecutables ELF directamente, sin conversión ni flags especiales. Migración transparente.

mosrun: Marca PCB como “migrable”. Scheduler monitorea carga y migra automáticamente. `mosrun -k 8 ./aplicacion` (-k maxjobs limita jobs concurrentes).

Herramientas:

Herramienta	Función
mosmon	Monitor en tiempo real: CPU, memoria, procesos/nodo, migraciones
mosps	Lista procesos en todos los nodos
mostat	Estadísticas: nodos activos, carga promedio, memoria total

Acceden via `/proc/hpc` (filesystem virtual procs).

SLURM: Puede funcionar con SLURM (60%+ de Top500). SLURM decide job/nodo; MOSIX optimiza recursos moviendo procesos. Integración no nativa, requiere configuración manual.

Limitaciones: Threads no migran automáticamente. Memoria compartida no soportada. Pipes/sockets funcionan; message queues/semaphores variables; shared memory no.

Comparación

Facilidad	Zephyr OS	MOSIX
Build system	West + CMake + Ninja	Standard gcc/Linux
Configuración	KConfig + Device Tree	mosrun + configuración de nodo
API del SO	Subconjunto POSIX	POSIX completa
Debugging	GDB + OpenOCD (JTAG/SWD)	GDB estándar Linux
Docs	Exhaustiva (docs.zephyrproject.org)	Limitada

9. Puertas Afuera

9.1 Difusión y Presencia

Aspecto	Zephyr OS	MOSIX
Adopción	3,000+ contribuidores en 70+ países	Inactiva desde 2017
Plataformas	1,000+ boards (ARM/RISC-V/x86/MIPS/ARC/SPARC)	Clusters universitarios históricos
Backing	Intel, Nordic, Renesas, NXP, Wind River	Ninguno (académico)
Eventos	Open Source Summit, Embedded World, Zephyr Developer Summit	Papers académicos (1998-2010)
Productos comerciales	Vestas, Google Chromebook, Oticon More, Framework Laptop, GARDENA	Cero casos modernos
Tendencia	69% organizaciones planea aumentar uso	Inactivo desde oct 2017

9.2 Soporte a Usuarios

Aspecto	Zephyr OS	MOSIX
Documentación	Completa y actualizada (docs.zephyrproject.org)	Desactualizada desde 2017 (FAQ, Admin Guide PDF, White Paper)
Comunidad	Discord (miles), GitHub Discussions, mailing lists	Prácticamente inactiva (<20 preguntas SO sin respuesta)
Soporte comercial	Disponible via miembros + Wind River Rocket (SLA)	No disponible
Seguridad	Security Subcommittee, parches backporteados a LTS (10-20 años)	Zero parches desde +8 años
Training	Training Partners autorizados (ModularMX, Goliath, Hacod)	No disponible

9.3 Casos de Uso

Dominio	Zephyr OS	MOSIX
IoT y sensotización industrial	[OK] Ideal (+1,000 boards, multi-protocolo)	[NO] Inactivo
Wearables y dispositivos médicos	[OK] Oticon More, HealthyPi Move (~4 KB)	[NO] No aplica
Microcontroladores de 32-bit	[OK] ARM Cortex-M/RISC-V/x86	[NO] No aplica

Dominio	Zephyr OS	MOSIX
Tiempo real embebido	[OK] Scheduler preemptive, tickless, deep sleep	[NO] No aplica
Clusters HPC académico	[NO] No es target	[WARNING] Histórico (1999-2010): genómica, CFD, meteorología
HPC producción	[NO] No es target	[NO] Inactivo → usar SLURM/Kubernetes

9.4 Costos y Licenciamiento

Aspecto	Zephyr OS	MOSIX
Licencia	Apache 2.0 (permisiva, sin copyleft, sin regalías)	Propietaria restrictiva (prohíbe modificación, ingeniería reversa)
Código fuente	Completo en GitHub	No disponible
Costo de entrada	\$0	Histórico: \$61,141 USD (año 2000, fuente: USENIX), actualmente sin precio público disponible (proyecto inactivo)
Costo por unidad	\$0 sin regalías	Desconocido (inactivo)
Soporte comercial	Opt-in via miembros, Wind River Rocket	No disponible
Viabilidad 2026	[OK] Activo con sponsors múltiples	[NO] Inactivo desde 2017, sin parches, restrictivo

10. Comparativa Técnica

Tabla Comparativa

Aspecto	Zephyr OS	MOSIX
Tipo	RTOS embebido	Cluster OS / HPC
Segmento	IoT, wearables, microcontroladores	HPC, grids académicos
Licencia	Apache 2.0 (permisiva)	Propietaria restrictiva
RAM mínima	~16 KB	N/A
Arquitectura	Híbrido monolítico unificado	Extensión de kernel Linux
Sistema de archivos	VFS con LittleFS, FAT FS, NVS	DFS (no tiene FS propio)

Aspecto	Zephyr OS	MOSIX
Gestión de memoria	MPU + Memory Domains + User Mode	Shared-nothing + Memory Ushering
Scheduling	Preemptive / Cooperative / Híbrido	Migración preemptiva entre nodos
Tiempo real	Sí (RTOS determinístico)	No
Migración de procesos	No aplica	Sí — transparente preemptiva
Seguridad	MPU + TrustZone + PSA Crypto + OpenSSF Gold	Sandboxing + checkpoint/restart (sin crypto)
Conectividad	BLE, Wi-Fi, Thread, 802.15.4, LoRa, Cellular, CAN	Ethernet, InfiniBand histórico
Herramientas	West, CMake, KConfig, Device Tree, QEMU	mosrun, mosmon, mosps, mostat
Comunidad	Activa (3,000+ contribuyentes, 2026)	Inactiva desde oct 2017
Soporte comercial	Sí (Nordic, Intel, NXP, Renesas, Wind River)	No
Última versión	LTS3 (2026)	MOSIX-4.4.4 (oct 2017)
Competidores reales	FreeRTOS, NuttX, RT-Thread, RIOT OS, ThreadX	SLURM, Kubernetes, OpenMPI, PBS Professional

Análisis

Comparación inherentemente “injusta”: No son competidores directos. Zephyr compite con FreeRTOS/NuttX (RTOS para microcontroladores); MOSIX competía con SLURM/PBS Professional (schedulers de cluster HPC).

Dimensión	Zephyr OS	MOSIX
Qué administra	Un microcontrolador individual	Un cluster de múltiples computadoras
Problema	Tiempo real en IoT embebido	HPC en clusters
Target hardware	Microcontroladores (4 KB - 2 MB RAM)	Clusters de PCs (64 GB - TB por nodo)
Escala	Un dispositivo	Cientos de nodos
Migración	No	Sí

Valor pedagógico: La comparación tiene valor académico, no para selección de producto. Ilustra cómo diferentes dominios generan soluciones radicalmente diferentes, y revela la importancia de factores no técnicos: gobernanza, licencia open source, comunidad activa vs. licencia propietaria y abandono.

Progresión histórica: MOSIX (1999-2017) → SLURM (2003-presente) → Kubernetes (2014-presente).

11. Conclusiones y Recomendaciones

11.1 Conclusiones

Zephyr OS (2026): Solución moderna, activa y bien respaldada para IoT embebido. 70% organizaciones en Norteamérica, 62% en Europa, 69% planeando aumentar uso.

Fortalezas: 1. **Gobernanza neutral multisponsor:** Linux Foundation con TSC (Nordic, Intel, NXP, Renesas, Wind River). Elimina vendor lock-in. 2. **Seguridad robusta para IoT regulado:** PSA Crypto API + mbedTLS, secure boot chains, secure storage, MPU con user mode, Security Subcommittee, OpenSSF Gold Badge (2018-2024). 3. **Conectividad wireless integrada:** BLE, Wi-Fi, Thread, 802.15.4, LoRa, Cellular, CAN bus en el kernel. 4. **Portabilidad extrema:** 1,000+ boards, 15+ arquitecturas CPU. Device Tree abstraes hardware. 5. **LTS para largo ciclo:** LTS3 estabilidad por años, ideal para productos industriales/médicos (10-20 años).

MOSIX: Enfoque académico interesante pero sin evolución desde 2017. Último release: MOSIX-4.4.4 (oct 2017).

Valor académico: 1. **Pionero en migración preemptiva (1977-presente):** Primero en demostrar funcionalmente migración preemptiva en clusters Linux (1999), 40+ años innovating. 2. **SSI completo:** Cluster como único sistema lógico — precursor del cloud computing moderno. 3. **Memory Ushering:** Algoritmo de migración proactiva antes de OOM, enseñado en cursos de sistemas distribuidos. 4. **Moraleja:** Evolución hacia SLURM/Kubernetes demuestra cómo soluciones pragmáticas superan a “perfectas pero frágiles”.

Inactivo: sin actualizaciones de seguridad, sin soporte comercial, sin compatibilidad con kernels modernos, licencia restrictiva que impidió contribución comunitaria.

11.2 Recomendaciones

1. Proyectos IoT/embebido: Zephyr OS [OK]

Por qué: Tiempo real determinístico, footprint mínimo (~16 KB), desarrollo activo (3,000+ contribuidores), gobernanza neutral, LTS para 10-20 años.

Casos: Productos IoT ciclos largos, dispositivos médicos/industriales regulados, múltiples protocolos wireless, portabilidad cross-vendor.

2. Estudio académico de clustering: MOSIX [BOOK]

Para aprender: Migración preemptiva de procesos, Single System Image (precursor de Kubernetes), Memory Ushering, por qué murió (licencia propietaria + falta de gobernanza = abandono).

3. NO usar MOSIX en producción [NO]

Razones: Abandonado desde 2017, sin security patches, sin soporte, sin seguridad moderna, propietario (no auditable, no extensible).

4. Matriz de Decisión

```
¿Real-time embebido con footprint mínimo?  
+-- Sí + ¿Sin MMU? → [OK] ZEPHYR  
\`-- No → ¿Para qué?  
    +-- HPC producción → [WRENCH] SLURM / Kubernetes  
    +-- Estudio académico → [BOOK] MOSIX  
    \`-- Otras necesidades → Alternativas según caso
```

Si necesitas...	Recomendación
Producto IoT comercial (2026+)	Zephyr OS
Seguridad robusta + conectividad	Zephyr OS
Largo ciclo vida producto industrial	Zephyr OS (LTS3)
Portabilidad cross-vendor	Zephyr OS
Prototipo rápido, equipo sin experiencia	FreeRTOS o RIOT OS
Aprender conceptos de clustering histórico	MOSIX (estudio)
Proyecto HPC real en 2026	SLURM o Kubernetes
Certificaciones pre-existentes (IEC 61508, ISO 26262)	ThreadX

11.3 Síntesis Final

“No existe ‘el mejor sistema operativo’ — existe ‘el correcto para tu problema’”

- **Zephyr para productos IoT reales en 2026:** comunidad activa, documentación completa, soporte comercial, seguridad robusta.
- **MOSIX para estudio académico:** valor histórico en migración preemptiva y SSI, no recomendado para producción moderno.

La viabilidad comercial depende tanto de factores organizacionales (governanza, comunidad, soporte) como de mérito técnico.

12. Bibliografía

Fuentes Oficiales Zephyr: - Zephyr Project Documentation: <https://docs.zephyrproject.org/> - Zephyr GitHub Repository: <https://github.com/zephyrproject-rtos/zephyr> - Zephyr Introduction (RAM 16KB): <https://docs.zephyrproject.org/latest/introduction/index.html> - Zephyr Security Overview: <https://docs.zephyrproject.org/latest/security/security-overview.html> - Linux Foundation Research “Zephyr at 10” (marzo 2026): <https://www.linuxfoundation.org/research/zephyr-turns-10> - OpenSSF Best Practices Gold Badge: <https://www.bestpractices.dev/projects/74/gold> (última actualización 2024-06-05)

Fuentes MOSIX: - MOSIX Official Site: <http://www.mosix.org/> - MOSIX Distributions & Licensing: https://mosix.cs.huji.ac.il/txt_distributions.html - Hebrew University CS Department: <https://www.cs.huji.ac.il/> - USENIX 2000 Historical Pricing (referencia obsoleta, link no disponible): “Historical price documented in USENIX 2000 proceedings”

Publicaciones: - Prof. Amnon Barak (~1,662 citas en sistemas distribuidos) - MOSIX Administrator's Guide (histórico) - MOSIX White Papers (histórico)

Recursos de Desarrollo: - Device Tree Specification - PSA Crypto API Documentation (Arm): <https://developer.arm.com/architectures/security-architectures/platform-security-architecture> - OpenAMP Framework

Información de Mercado y Competidores: - Linux Foundation Members List (2025-2026): <https://www.linuxfoundation.org/members> - SLURM Workload Manager: <https://slurm.schedmd.com/> - Top500 Supercomputers List: <https://www.top500.org/>

Nota: Todas las fuentes web consultadas en mayo 2026.

Documento elaborado para el Trabajo Práctico Especial de Fundamentos de Sistemas Operativos — Universidad Nacional de Mar del Plata — Mayo 2026. Grupo: ARRIAGA, Mario Esteban; BELLONE, Martín; BISCAY, Federico Javier; CALLA ALIENDE, Federico; CARDOZO, Lucas.